

Protocoles interactifs.

Deux machines discutent : le *prouveur* $\mathcal{P}$ et le *vérificateur* $\mathcal{V}$. Le prouveur a une puissance de calcul illimitée, et le vérificateur ne fait pas initialement « confiance » au prouveur.

☞ **Exemple 1.** On donne un protocole interactif pour SAT.

Algorithme 1 Protocole interactif pour SAT

Vérificateur $\mathcal{V}$	Prouveur $\mathcal{P}$
	1 : Calculer $(a_1, \dots, a_n)$ une assignation qui satisfait F .
	$\xleftarrow{a_1, \dots, a_n}$
1 : si $F(a_1, \dots, a_n) = 1$ alors	
2 : Accepter	
3 : sinon	
4 : Rejeter	

Analysons ce protocole :

- ▷ si $F \notin \text{SAT}$ alors $\mathcal{V}$ rejette toujours, peu importe $\tilde{\mathcal{P}}$;
- ▷ si $F \in \text{SAT}$ alors $\mathcal{P}$ peut convaincre $\mathcal{V}$ d'accepter.

☞ **Exemple 2.** On donne un protocole interactif pour le *non-isomorphisme de graphes*.

On définit le problème **ISOGRAPH** :

ISOGRAPH

Entrée. Deux graphes G_0 et G_1 **Sortie.** Est-ce que G_0 est isomorphe à G_1 ?

où un *isomorphisme* entre (W, E) et $(W, E') = \pi(W, E)$ est une permutation $\pi : W \rightarrow W$ des sommets avec

$$E' := \{ (i, j) \mid (\pi(i), \pi(j)) \in E \}.$$

On notera alors $(W, E) \cong (W, E')$.

On a que **ISOGRAPH** $\in$ **NP** (avec π le certificat). Mais, on ne sait pas si le problème est **NP**-complet, il existe un algorithme en temps $n^{O(\log n)}$. (Par définition, le problème de *non*-isomorphisme de graphe est dans **coNP**.)

On donne un protocole interactif pour le problème de non-isomorphisme de graphe. En entrée, G_0, G_1 est une paire de graphes sur un même ensemble de sommets W .

Le but de $\mathcal{P}$ est de convaincre $\mathcal{V}$ que $G_0 \not\cong G_1$.

- ▷ Si $G_0 \not\cong G_1$ alors G' est isomorphe à un unique graphe $G_{b'}$ et renvoie b' au vérificateur.
- ▷ Si $G_0 \cong G_1$, alors pour tout prouveur, $\Pr[\mathcal{V} \text{ accepte}] = 1/2$. En effet, pour tout G' ,

$$\Pr[\tilde{\mathcal{P}} \text{ renvoie } G' \mid b = 0] = \Pr[\tilde{\mathcal{P}} \text{ renvoie } G' \mid b = 1],$$

notons $p_{i,j} := \Pr[\tilde{\mathcal{P}} \text{ envoie } b' = j \mid b = i]$ et on observe que $p_{11} = p_{21}$, d'où

$$\Pr[\mathcal{V} \text{ accepte}] = \frac{1}{2}(p_{11} + p_{22}) = \frac{1}{2}(p_{21} + p_{22}) = \frac{1}{2}.$$

Algorithme 2 Protocole interactif pour $\Sigma^* \setminus \text{ISOGRAPH}$

Vérificateur $\mathcal{V}$	Prouveur $\mathcal{P}$
Entrée G_0, G_1	Entrée G_0, G_1
1 : Choisir un bit $b \in \{0, 1\}$ aléatoire	
2 : Choisir une permutation π de W aléatoire	
3 : Calculer $G' := \pi(G_b)$	
	$\xrightarrow{G'}$ 1 : si $G_0 \not\cong G_1$ alors 2 : Le graphe G' est iso- morphe à un unique $G_{b'}$. 3 : sinon 4 : Soit $b' \in \{0, 1\}$ choisi aléatoirement.
	$\xleftarrow{b'}$
4 : si $b = b'$ alors	
5 : Accepter	
6 : sinon	
7 : Rejeter	

On peut réduire l'erreur en répétant le protocole k fois, et on accepte si (G_0, G_1) est toujours accepté par $\mathcal{V}$, et on aura $\Pr[\mathcal{V} \text{ accepte}] \leq 1/2^k$.

1 La classe IP.

On a la situation représentée en figure 1, où

- ▷ le x est l'entrée du problème (de taille n) ;
- ▷ le r est la donnée des choix aléatoires de v (de taille polynomiale en n) ;
- ▷ les y_i sont les messages de $\mathcal{V}$ à $\mathcal{P}$ (de taille polynomiale en n) ;
- ▷ les z_i sont les messages de $\mathcal{P}$ à $\mathcal{V}$ (de taille polynomiale en n) ;
- ▷ le s est la longueur de l'échange (avec s polynomial en n).

La puissance de calcul de $\mathcal{P}$ est *non-bornée*, et celle de $\mathcal{V}$ est polynomialement bornée. La *décision finale du vérificateur* est $\mathcal{V}(x, r, z_1, \dots, z_s) \in \{0, 1\}$.

Vérificateur
 $\mathcal{V}$

Prouveur
 $\mathcal{P}$

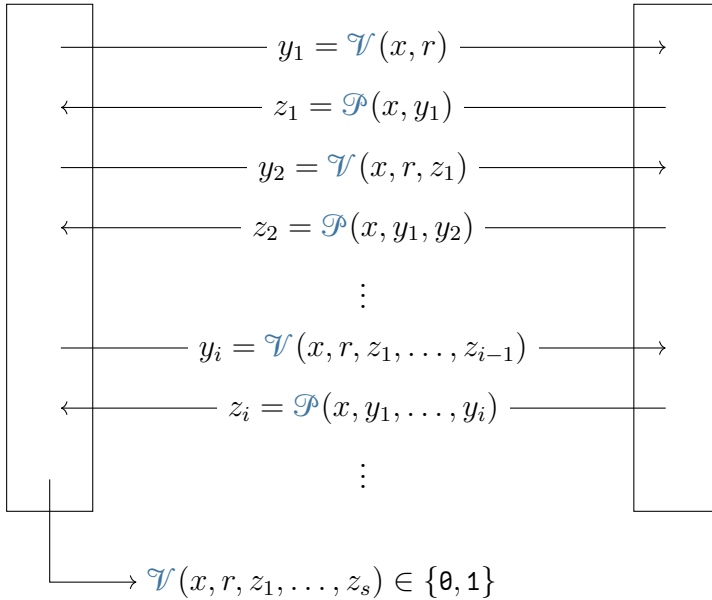


Figure 1 | *Protocole interactif*

Définition 1. On définit $\mathbf{IP}$ la classe des langages que l'on peut reconnaître à l'aide d'un protocole interactif avec probabilité d'erreur inférieur à $1/3$. Autrement dit, un langage L est dans $\mathbf{IP}$ ss'il existe un vérificateur $\mathcal{V}$ et un prouveur $\mathcal{P}$ tels que

1. si $x \in L$ alors $\Pr[\mathcal{P} \text{ pousse } \mathcal{V} \text{ à accepter}] \geq \frac{2}{3}$;

2. si $x \notin L$ alors pour tout prouveur $\tilde{\mathcal{P}}$,

$$\Pr[\tilde{\mathcal{P}} \text{ pousse } \mathcal{V} \text{ à accepter}] \leq \frac{1}{3}.$$

▮ **Exemple 3.** On a $\text{SAT} \in \text{IP}$ et $\Sigma^* \setminus \text{ISOGRAPH} \in \text{IP}$.

▮ **Théorème 1.** On a $\text{IP} = \text{PSPACE}$.

▮ **Remarque 1.** On a $\text{IP} \subseteq \text{PSPACE}$, c'est le « sens facile ».

En revanche, le résultat $\text{PSPACE} \subseteq \text{IP}$ *ne se relativise pas* : il existe A tel que PSPACE^A n'est pas inclus dans IP^A (et même coNP^A n'est pas inclus dans IP^A).

Pour montrer $\text{PSPACE} \subseteq \text{IP}$, on utilise le fait que QBF est PSPACE -complet : il suffira donc de montrer que $\text{QBF} \in \text{IP}$, et on pourra utiliser les propriétés spécifiques de QBF, en particulier l'*arithmétisation* (on peut voir des formules booléennes comme des polynômes).

▮ **Définition 2.** On définit le problème $\#\text{SAT}_D$ par

$\#\text{SAT}_D$	Entrée. Une formule booléenne ϕ sous CNF et $k \in \mathbb{N}$ Sortie. La formule ϕ est-elle satisfaite par exactement k valuations ?
------------------	---

ou, de manière équivalente,

$$\#\text{SAT}_D := \left\{ \langle \phi, k \rangle \mid \begin{array}{l} \phi \text{ est une formule booléenne satisfaite} \\ \text{par exactement } k \text{ assignations} \end{array} \right\}.$$

▮ **Théorème 2.** On a $\#\text{SAT}_D \in \text{IP}$.

▮ **Preuve.** La preuve se base sur la technique d'*arithmétisation* : on associe à $\phi(x_1, \dots, x_n)$ un polynôme $P_\phi(x_1, \dots, x_n)$ que l'on définit par induction :

- ▷ si $\phi = x$, alors $P_\phi := x$;
- ▷ si $\phi = \psi \wedge \vartheta$, alors $P_\phi := P_\psi \times P_\vartheta$;
- ▷ si $\phi = \neg\psi$ alors $P_\phi := 1 - P_\psi$;
- ▷ si $\phi = \psi \vee \vartheta$, alors

$$P_\phi := P_\psi + P_\vartheta - P_\psi \times P_\vartheta = 1 - (1 - P_\psi) \times (1 - P_\vartheta).$$

On définit également, pour $i \in \llbracket 0, m \rrbracket$,

$$f_i(x_1, \dots, x_i) := \sum_{(a_{i+1}, \dots, a_n) \in \{0,1\}^{n-i}} P_\phi(x_1, \dots, x_i, a_{i+1}, \dots, a_n).$$

En particulier, $f_n(x_1, \dots, x_n) = P_\phi(x_1, \dots, x_n)$. Et, de plus, f_0 est une constante qui compte le nombre d'instances qui vont satisfaire la formule ϕ . On a

$$\deg f_i \leq \deg P_\phi \leq |\phi|,$$

et si $f \neq g$ avec $\deg f \leq d$ et $\deg g \leq d$ alors

$$\Pr[f(x) = g(x)] \leq \frac{d}{|S|}, \quad (*)$$

si x est choisi uniformément dans S .

Maintenant, décrivons le protocole.

Algorithme 3 Protocole interactif pour $\sharp\text{SAT}_D$

Vérificateur $\mathcal{V}$	Prouveur $\mathcal{P}$
	1 : Calculer des polynômes $f_0, \dots, f_n$
	$\xleftarrow{f_0}$
1 : si $f_0 \neq k$ alors	
2 : $\perp$ Rejeter	$\xleftarrow{f_1(z)}$
3 : $y_0 := f_1(0) + f_1(1)$	
4 : si $f_0 \neq y_0$ alors	
5 : $\perp$ Rejeter	
6 : Choisir $r_1 \in S$ aléatoire- ment	$\xrightarrow{r_1}$
	2 : Calculer $f_2(r_1, z)$, polynôme en z
	$\xleftarrow{f_2(r_1, z)}$
7 : $y_1 := f_2(r_1, 0) + f_2(r_1, 1)$	
8 : si $f_1(r_1) \neq y_1$ alors	
9 : $\perp$ Rejeter	
10 : Choisir $r_2 \in S$ aléatoire- ment	$\xrightarrow{r_2}$
	3 : Calculer $f_3(r_1, r_2, z)$, poly- nôme en z
	$\xleftarrow{f_3(r_1, r_2, z)}$
11 : <i>etc</i>	4 : <i>etc</i>
12 : $y_n := f_n(r_1, \dots, r_n)$	
13 : Accepter si $P_\phi(r_1, \dots, r_n) = y_n$	
14 : Rejeter sinon	

Dans un premier temps, analysons la complétude du protocole. Si $\langle \phi, k \rangle \in \sharp\text{SAT}_D$, montrons qu'il existe $\mathcal{P}$ qui va convaincre $\mathcal{V}$ à coup sûr. Pour cela, il suffit de prendre $\mathcal{P}$ qui respecte le protocole

avec les f_i définies précédemment.

Maintenant, regardons la consistance du protocole. Supposons que $\langle \phi, k \rangle \notin \#SAT_D$, i.e., k n'est pas le nombre d'assignations qui vont satisfaire ϕ . Montrons que tout prouveur $\tilde{\mathcal{P}}$ a probabilité inférieure à $1/3$ de convaincre $\mathcal{V}$. Lors du protocole, $\tilde{\mathcal{P}}$ va envoyer des polynômes $\tilde{f}_i$ qui peuvent être différents des f_i . On dira que $\tilde{\mathcal{P}}$ est *chanceux à la phase i* si

$$f_i(r_1, \dots, r_{i-1}, z) \neq \tilde{f}_i(r_1, \dots, r_{i-1}, z)$$

mais que $\mathcal{V}$ choisit r_i tel que

$$f_i(r_1, \dots, r_i) = \tilde{f}_i(r_1, \dots, r_i).$$

▮ **Lemme 1.** Si $\langle \phi, k \rangle \notin \#SAT_D$ et si $\tilde{\mathcal{P}}$ n'est jamais chanceux alors $\mathcal{V}$ rejette.

▮ **Preuve.** On a que $k = \tilde{f}_0 \neq f_0$ sinon $\mathcal{V}$ rejette immédiatement. De même, $\tilde{f}_0 = \tilde{f}_1(0) + \tilde{f}_1(1)$ sinon $\mathcal{V}$ rejette à la phase suivante. On a nécessairement que $\tilde{f}_1 \neq f_1$ car, sinon, on aurait

$$\tilde{f}_0 = \tilde{f}_1(0) + \tilde{f}_1(1) = f_1(0) + f_1(1) = f_0.$$

Puisque $\tilde{\mathcal{P}}$ n'est pas chanceux, $\mathcal{V}$ va choisir une valeur de r_1 telle que $\tilde{f}_1(r_1) \neq f_1(r_1)$. On peut raisonner ainsi jusqu'à la fin du protocole. Lors de la dernière phase, on a donc

$$\tilde{f}_n(r_1, \dots, r_n) \neq f_n(r_1, \dots, r_n),$$

et, en particulier, $\tilde{f}_n(r_1, \dots, r_n) \neq P_\phi(r_1, \dots, r_n)$. Ainsi, $\mathcal{V}$ va finalement rejeter (s'il n'avait pas rejeté avant). $\square$

▮ **Lemme 2.** On a

$$\Pr[\exists i, \tilde{\mathcal{P}} \text{ est chanceux à l'étape } i] \leq \frac{n \times |\phi|}{|S|}.$$

▮ **Preuve.** Pour $i \in \llbracket 1, n \rrbracket$, on a par (*) que

$$\Pr[\tilde{\mathcal{P}} \text{ est chanceux à l'étape } i] \leq \frac{|\phi|}{|S|},$$

et il suffit juste d'appliquer la borne de l'union. □

Désormais, il ne reste plus qu'à analyser la complexité pour vérifier que le vérificateur est bien en temps polynomial. Par la suite, on supposera que les polynômes sont à coefficients entiers. On peut vérifier que les polynômes échangés ont des coefficients de taille polynomiale. Sinon, on peut se placer sur un corps fini $\mathbb{F}$ dans lequel les coefficients sont nécessairement de taille bornée. La seule contrainte sur $\mathbb{F}$ est $|\mathbb{F}| \geq |S|$.

Le prouveur $\mathcal{P}$ peut envoyer p tel que $\mathbb{F} = \mathbb{Z}/p\mathbb{Z}$ à $\mathcal{V}$ qui va vérifier que p est premier, avant toute chose, ce qui se fait en temps polynomial.¹ Pour tout S , on a qu'il existe un nombre premier p avec $|S| \leq p \leq 2|S|$ qui peut être renvoyé par $\mathcal{P}$ (*postulat de Bertrand*).

Il ne reste plus qu'à représenter P_ϕ . Puisque le représenter avec des monômes peut être exponentiel (par exemple avec la formule $\phi = \bigwedge_{i=1}^n (\neg x_i)$), on peut le représenter avec un circuit arithmétique ou par une formule arithmétique (*i.e.* un arbre). □

2 Preuve de $\text{IP} \subseteq \text{PSPACE}$.

Soit $L \in \text{IP}$ et soit $\mathcal{V}$ le vérificateur associé.

1. Notamment avec le test Agrawal–Kayal–Saxena.

Lemme 3. On peut calculer, sur une entrée x , en espace polynomial

$$\max_{\mathcal{P}} \Pr[\mathcal{P} \text{ pousse } \mathcal{V} \text{ à accepter } x].$$

Preuve. On suppose que sur une entrée $x \in \{0, 1\}^n$,

- ▷ le nombre de « phases » est exactement $s := s(n)$;
- ▷ le vérificateur $\mathcal{V}$ utilise exactement $r := r(n)$ bits aléatoires à chaque phase ;
- ▷ le prouveur $\mathcal{P}$ envoie des messages de longueurs exactement $\ell := \ell(n)$ à chaque phase.

On peut représenter cette situation comme un arbre.

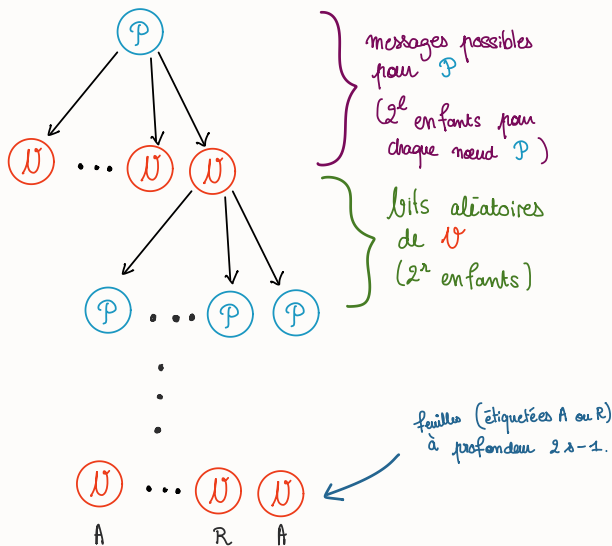


Figure 2 | Arbre permettant de calculer le résultat

En chaque nœud on a une histoire partielle H qui correspond à la liste des messages envoyés par $\mathcal{P}$ et des bits aléatoires tirés par

$\mathcal{V}$. La *valeur* d'un nœud est

$$\max_{\mathcal{P}} \Pr[\mathcal{P} \text{ pousse } \mathcal{V} \text{ à accepter } x].$$

Pour une feuille, sa valeur est 0 ou 1.

1. Pour un nœud $\mathcal{P}$, on a

$$\Pr[\mathcal{V} \text{ accepte } x \mid H] = \max_{m \in \{0,1\}^n} \Pr[\mathcal{V} \text{ accepte } x \mid H \# m].$$

2. Pour un nœud $\mathcal{V}$, on a

$$\Pr[\mathcal{V} \text{ accepte } x \mid H] = \frac{1}{2^r} \sum_{y \in \{0,1\}^r} \Pr[\mathcal{V} \text{ accepte } x \mid H \# y].$$

On évalue la racine avec un algorithme récursif, avec une alternance de max et de moyenne. $\square$

▮ **Corollaire 1.** On peut toujours supposer que $\mathcal{P}$ fonctionne avec un espace polynomial. $\square$